



REPÚBLICA DE ANGOLA
MINISTÉRIO DA EDUCAÇÃO
[NOME DA INSTITUIÇÃO DE ENSINO]

SGFE - SISTEMA DE GESTÃO DAS FINANÇAS DO ESTADO

Plataforma digital para gestão, execução, monitorização, relatórios e auditoria das finanças públicas

Autor(es): [NOME DO ESTUDANTE / GRUPO]
N.º de Processo: [N.º]
Curso: [CURSO]
Orientador: [NOME DO ORIENTADOR]

LUANDA, 2026

Folha de Rosto

[NOME DO ESTUDANTE / GRUPO]

SGFE - SISTEMA DE GESTAO DAS FINANÇAS DO ESTADO

Trabalho de Conclusão de Curso apresentado como requisito parcial para a obtenção do grau de Técnico Médio em [CURSO], na [NOME DA INSTITUIÇÃO DE ENSINO].

Orientador: [NOME DO ORIENTADOR]

LUANDA, 2026

Folha de Aprovacao

SGFE - SISTEMA DE GESTAO DAS FINANÇAS DO ESTADO

Nome do Estudante	N.o Processo	Nota T.P	Nota Relat.	Nota Def.	Nota Final
[Nome 1]	[N.o]				
[Nome 2]	[N.o]				

Observacoes:

Corpo de Jurado

Funcao	Assinatura
1.o Vogal	
2.o Vogal	
Presidente da Mesa do Juri	
Professor Orientador	
Coordenador do Curso	

Apresentado aos ____ de _____ de 2026.

Dedicatoria

Dedicamos este trabalho a todos os que acreditam que a tecnologia pode tornar a gestao publica mais clara, segura e eficiente.

Aos nossos pais e encarregados de educacao, pelo apoio constante, pela paciencia e pelo sacrificio que permitiram a continuidade da nossa formacao.

Aos professores e orientadores, pela exigencia tecnica, pelas correcções e pelo incentivo a desenvolver uma solucao que nao se limita a funcionar, mas que procura responder a um problema real da administracao publica.

E a todos os profissionais que diariamente trabalham na gestao das finanças do Estado, este projecto e uma contribuicao para tornar o seu trabalho mais organizado, rastreavel e confiavel.

Agradecimentos

Agradecemos primeiramente a Deus, pela vida, pela saude e pela forca necessaria para ultrapassar as dificuldades encontradas ao longo do desenvolvimento do projecto.

Ao nosso orientador, [NOME DO ORIENTADOR], pela disponibilidade, pelas recomendacoes tecnicas e pelo acompanhamento durante as fases de analise, desenvolvimento, testes e redacao deste relatorio.

Aos docentes da [NOME DA INSTITUICAO DE ENSINO], pelo conhecimento transmitido ao longo do curso, especialmente nas areas de programacao, bases de dados, redes, seguranca, analise de sistemas e engenharia de software.

Aos colegas, familiares e amigos que colaboraram com ideias, testes e observacoes sobre a experiencia de utilizacao da plataforma.

Por fim, agradecemos a todos os que directa ou indirectamente contribuiram para que o SGFE deixasse de ser apenas uma ideia e se transformasse numa solucao funcional, documentada e preparada para evoluir.

Resumo

O presente relatório descreve o desenvolvimento do SGFE - Sistema de Gestão das Finanças do Estado, uma plataforma web concebida para apoiar a gestão, execução, monitorização, relatórios e auditoria das finanças públicas. O sistema foi desenhado para responder a problemas frequentes no controlo financeiro institucional, como dispersão de informação, baixa rastreabilidade de operações, fragilidade no acompanhamento do tecto orçamental e dificuldade de consolidar receitas, despesas e auditoria num único ambiente digital.

O SGFE organiza o domínio financeiro em torno de Unidades Orçamentais, utilizadores, orçamentos, classificações económicas, receitas RUPE, despesas, relatórios e logs de auditoria. A solução implementa perfis de acesso por RBAC, nomeadamente ADMIN, GESTOR e AUDITOR, garantindo que cada utilizador actua apenas dentro das permissões atribuídas. O backend foi desenvolvido em Java 21 com Spring Boot, Spring Security, Spring Data JPA, Flyway, MySQL 8, JWT e refresh tokens armazenados de forma segura. O frontend foi construído em Next.js, React, TypeScript, Tailwind CSS, TanStack Query, TanStack Table, React Hook Form, Zod e Recharts.

A arquitectura do sistema privilegia separação de responsabilidades, validação de regras de negócio no backend, persistência relacional com integridade referencial, proteções de segurança, auditoria persistente e exportação de relatórios em PDF e Excel. O projecto inclui ainda mecanismos de recuperação de palavra-passe, cookies HttpOnly, rotação de refresh token, cabeçalhos de segurança, CORS configurável, protecção contra pedidos suspeitos e testes automatizados.

Como resultado, foi obtida uma plataforma funcional, moderna e institucional, capaz de centralizar processos financeiros, reduzir riscos operacionais e aumentar a transparência da gestão pública. O SGFE apresenta-se como uma base sólida para evolução futura, podendo receber integrações oficiais, assinaturas digitais, dashboards avançados, notificações, segregação de funções e mecanismos adicionais de controlo.

Palavras-chave: SGFE; Finanças Públicas; Unidade Orçamental; RUPE; Orçamento; Auditoria; Spring Boot; Next.js; MySQL; JWT; RBAC.

Abstract

This report describes the development of SGFE - State Finance Management System, a web platform designed to support the management, execution, monitoring, reporting and auditing of public finances. The system was created to address common issues in institutional financial control, such as fragmented information, limited traceability of operations, weak budget ceiling monitoring and difficulty consolidating revenue, expenditure and audit data in a single digital environment.

SGFE structures the financial domain around Budgetary Units, users, budgets, economic classifications, RUPE revenues, expenditures, reports and audit logs. The solution implements role-based access control with ADMIN, MANAGER and AUDITOR profiles, ensuring that each user acts only within the permissions assigned to them. The backend was developed with Java 21, Spring Boot, Spring Security, Spring Data JPA, Flyway, MySQL 8, JWT and securely stored refresh tokens. The frontend was built with Next.js, React, TypeScript, Tailwind CSS, TanStack Query, TanStack Table, React Hook Form, Zod and Recharts.

The system architecture prioritises separation of concerns, backend enforcement of business rules, relational persistence with referential integrity, security protections, persistent auditing and report export in PDF and Excel formats. The project also includes password recovery, HttpOnly cookies, refresh token rotation, security headers, configurable CORS, protection against suspicious browser requests and automated tests.

As a result, a functional, modern and institutional platform was achieved, capable of centralising financial processes, reducing operational risks and increasing transparency in public management. SGFE provides a solid foundation for future evolution, including official integrations, digital signatures, advanced dashboards, notifications, segregation of duties and additional control mechanisms.

Keywords: SGFE; Public Finance; Budgetary Unit; RUPE; Budget; Audit; Spring Boot; Next.js; MySQL; JWT; RBAC.

Epigrafe

“A excelencia de um sistema publico nao se mede apenas pela tecnologia que usa,
mas pela confianca que consegue produzir.”

Lista de Siglas e Abreviaturas

Sigla	Significado
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DTO	Data Transfer Object
FK	Foreign Key
HSTS	HTTP Strict Transport Security
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
MER	Modelo Entidade-Relacionamento
MVC	Model-View-Controller
NCD	Nota de Cabimentacao de Despesa
NLD	Nota de Liquidacao de Despesa
OGE	Orcamento Geral do Estado
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
RUPE	Referencia Unica de Pagamento ao Estado
SGFE	Sistema de Gestao das Financas do Estado
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
UO	Unidade Orcamental
UX	User Experience
XSS	Cross-Site Scripting

Lista de Figuras

- Figura 1 - Arquitectura logica do SGFE
- Figura 2 - Fluxo de autenticacao e renovacao de sessao
- Figura 3 - Modelo relacional simplificado
- Figura 4 - Fluxo da receita RUPE
- Figura 5 - Fluxo da despesa publica

Lista de Tabelas

- Tabela 1 - Requisitos funcionais do SGFE
- Tabela 2 - Requisitos nao funcionais do SGFE
- Tabela 3 - Requisitos de interface
- Tabela 4 - Perfis e responsabilidades
- Tabela 5 - Tecnologias utilizadas
- Tabela 6 - Matriz de permissoes
- Tabela 7 - Principais tabelas da base de dados
- Tabela 8 - Endpoints principais da API REST
- Tabela 9 - Plano de evolucao futura

Indice Geral

- 1 Introducao
 - 1.1 Consideracoes Iniciais
 - 1.2 Justificativa do Tema
 - 1.3 Problematica
 - 1.4 Objectivos
 - 1.5 Hipoteses
 - 1.6 Estrutura do Relatorio
- 2 Fundamentacao Teorica
 - 2.1 Definicao dos Termos
 - 2.2 Requisitos do Sistema
- 3 Metodologia
 - 3.1 Tipo de Pesquisa
 - 3.2 Universo e Amostra
 - 3.3 Tecnicas de Recolha de Dados
 - 3.4 Instrumentos de Recolha de Dados
 - 3.5 Metodologia de Desenvolvimento
 - 3.6 Delimitacao do Tema
- 4 Analise e Apresentacao dos Resultados
 - 4.1 Caracterizacao do Projecto
 - 4.2 Missao, Visao e Valores
 - 4.3 Organograma Funcional do Projecto
 - 4.4 Tecnologias e Ferramentas
 - 4.5 Arquitectura do Sistema
 - 4.6 Modelo de Dados
- 5 Desenvolvimento dos Modulos do Sistema
 - 5.1 Gestao de Utilizadores e Identidade
 - 5.2 Unidades Orcamentais
 - 5.3 Orcamentos e Tecto Orcamental
 - 5.4 Classificacoes Economicas
 - 5.5 Receitas RUPE
 - 5.6 Despesas
 - 5.7 Dashboard
 - 5.8 Relatorios
 - 5.9 Auditoria
 - 5.10 Seguranca
 - 5.11 Estudo de Viabilidade
 - 5.12 Resultados Obtidos
- 6 Conclusao
- 7 Perspectivas Futuras
- 8 Referencias Bibliograficas
- 9 Apendices

1 Introducao

1.1 Consideracoes Iniciais

A gestao das finanças públicas exige rigor, transparência, rastreabilidade e capacidade de resposta. Em qualquer administração moderna, os processos ligados ao orçamento, a arrecadação de receitas, a execução de despesas, a produção de relatórios e a auditoria precisam de informação consistente e acessível no momento certo.

No contexto institucional, a ausência de uma plataforma integrada pode provocar duplicação de registos, perda de histórico, dificuldade de fiscalização, cálculos inconsistentes de saldo e baixa visibilidade sobre a execução do orçamento. Estes problemas afectam não apenas a eficiência operacional, mas também a confiança nos processos administrativos.

O SGFE surge como uma solução tecnológica orientada ao domínio das finanças públicas. A plataforma permite organizar Unidades Orçamentais, utilizadores, classificações económicas, orçamentos, receitas RUPE, despesas, relatórios e auditoria num único ambiente web. O objectivo não é substituir a responsabilidade humana, mas oferecer uma ferramenta que reduza erro, aumente controlo e torne cada operação verificável.

A solução foi desenvolvida com uma arquitectura moderna, composta por frontend em Next.js e backend em Spring Boot. As regras críticas ficam no backend, a base de dados MySQL garante integridade relacional, e o frontend oferece uma experiência de utilização profissional, responsiva e adequada a um sistema institucional.

1.2 Justificativa do Tema

O tema justifica-se pela importância de digitalizar processos financeiros públicos com segurança e rastreabilidade. Sistemas de gestão financeira não podem depender apenas de folhas de cálculo, registos isolados ou processos manuais, pois estas abordagens aumentam o risco de inconsistência, perda de dados e dificuldade de auditoria.

O SGFE responde a uma necessidade real: centralizar a execução financeira por Unidade Orçamental, garantindo que receitas, despesas e orçamentos estejam ligados a entidades, utilizadores e classificações económicas. Esta centralização permite gerar relatórios mais fiáveis e acompanhar o tecto orçamental com maior clareza.

A escolha do tema também se justifica pelo seu valor pedagógico. O projecto aplica conceitos essenciais de engenharia de software, bases de dados relacionais, APIs REST, segurança, RBAC, auditoria, frontend moderno, validação de formulários, testes automatizados e arquitectura em camadas.

1.3 Problemática

Antes da existência de um sistema integrado, a gestão financeira institucional pode sofrer com vários problemas:

- informação financeira dispersa entre diferentes ferramentas;
- dificuldade de saber quem registou ou alterou determinada operação;
- falta de controlo consistente do saldo orçamental;
- baixa capacidade de produzir relatórios consolidados;

- risco de acesso indevido a áreas sensíveis;
- ausência de separação clara entre administração, gestão e auditoria;
- dificuldade de preservar histórico financeiro quando entidades são alteradas;
- dependência de validações apenas na interface, sem garantia no backend.

Diante destas limitações, a pergunta central do projecto é:

Como desenvolver uma plataforma digital segura, moderna e rastreável que permita gerir Unidades Orcamentais, orçamentos, receitas RUPE, despesas, relatórios e auditoria, garantindo controlo por perfil e integridade dos dados financeiros?

1.4 Objectivos

1.4.1 Objectivo Geral

Desenvolver uma plataforma web para gestão das finanças do Estado, capaz de centralizar Unidades Orcamentais, utilizadores, orçamentos, receitas RUPE, despesas, relatórios e auditoria, com segurança por perfil, regras de negócio no backend e persistência relacional integrada.

1.4.2 Objectivos Específicos

- implementar autenticação segura com JWT, refresh token e cookies HttpOnly;
- definir perfis de acesso para ADMIN, GESTOR e AUDITOR;
- permitir a gestão de Unidades Orcamentais e seus responsáveis;
- gerir utilizadores, perfis, estados e associação institucional;
- cadastrar classificações económicas para receitas e despesas;
- registar orçamentos por UO e ano fiscal;
- controlar receitas RUPE com geração automática de código único;
- suportar autorização para receitas retroactivas com intervenção do auditor;
- registar despesas e acompanhar estados financeiros;
- calcular execução, saldos e indicadores no dashboard;
- gerar relatórios em PDF e Excel;
- guardar logs de auditoria com utilizador, UO, acção, resultado e contexto;
- aplicar validações de integridade no backend e na base de dados;
- desenvolver interface moderna, responsiva e profissional;
- validar o sistema com testes automatizados e build de produção.

1.5 Hipoteses

Hipótese 1: A centralização dos processos financeiros numa plataforma única reduz a fragmentação da informação e melhora a capacidade de auditoria.

Hipótese 2: A aplicação de RBAC, logs persistentes, cookies HttpOnly, refresh token rotativo e regras de negócio no backend aumenta a segurança operacional do sistema.

Hipótese 3: Uma interface moderna, clara e responsiva melhora a experiência dos utilizadores e facilita a adopção do sistema por perfis administrativos, gestores e auditores.

1.6 Estrutura do Relatório

O relatório está organizado em nove capítulos. O Capítulo 1 apresenta a introdução, justificativa, problemática, objectivos e hipóteses. O Capítulo 2 desenvolve a fundamentação teórica. O Capítulo 3 descreve a metodologia de investigação e desenvolvimento. O Capítulo 4 apresenta a análise do projecto, tecnologias, arquitectura e modelo de dados. O Capítulo 5 detalha os módulos implementados. O Capítulo 6 apresenta a conclusão. O Capítulo 7 descreve perspectivas futuras. O Capítulo 8 apresenta as referências bibliográficas. O Capítulo 9 reúne apêndices técnicos, endpoints e configuração do ambiente.

2 Fundamentacao Teorica

2.1 Definicao dos Termos

Financas Publicas: area da administracao que estuda e organiza a arrecadacao, alocao, execucao e controlo dos recursos financeiros do Estado.

Unidade Orcamental: entidade institucional que recebe dotacao orcamental e executa receitas ou despesas dentro de um determinado exercicio fiscal.

Orcamento Geral do Estado: instrumento de planeamento financeiro que estima receitas e fixa despesas publicas para um periodo fiscal.

Tecto Orcamental: limite financeiro atribuido a uma Unidade Orcamental, servindo como base para controlo de execucao.

Cabimentacao: fase em que uma despesa e reservada no orcamento, reduzindo a disponibilidade do tecto para novas despesas.

Liquidacao: fase em que se confirma que a obrigacao financeira pode ser reconhecida para pagamento, apos validacao do direito do credor.

Pagamento: fase final da despesa, em que a obrigacao liquidada e efectivamente paga.

RUPE: Referencia Unica de Pagamento ao Estado, usada no SGFE como codigo numerico unico de receita.

RBAC: modelo de autorizacao baseado em papeis, em que cada perfil possui permissoes definidas.

JWT: padrao de token assinado que transporta informacoes de identidade e autorizacao entre cliente e servidor.

Refresh Token: token de longa duracao usado para renovar a sessao sem exigir novo login constante.

API REST: estilo de comunicacao entre sistemas usando HTTP, recursos, metodos e respostas padronizadas.

Auditoria: registo sistematico de operacoes relevantes, permitindo identificar quem realizou uma acao, quando, sobre qual entidade e com que resultado.

MySQL: sistema de gestao de base de dados relacional utilizado para persistir as informacoes do SGFE.

Spring Boot: framework Java usado para criar APIs robustas, seguras e modulares.

Next.js: framework React utilizado para construir interfaces web modernas, com rotas, renderizacao e optimizacoes integradas.

2.2 Requisitos do Sistema

2.2.1 Requisitos Funcionais

N.o	Area	Descricao
RF-001	Autenticacao	O sistema deve permitir login por email e palavra-passe.
RF-002	Autenticacao	O sistema deve emitir access token e refresh token em cookies HttpOnly.
RF-003	Autenticacao	O sistema deve permitir refresh de sessao com rotacao de token.
RF-004	Autenticacao	O sistema deve permitir logout com revogacao de refresh token.
RF-005	Utilizadores	O sistema deve permitir gerir utilizadores, perfis, estados e UO.
RF-006	UO	O sistema deve permitir criar, listar, editar e remover Unidades Orcamentais.
RF-007	Orcamentos	O sistema deve permitir cadastrar tecto por UO e ano fiscal.
RF-008	Classificacoes	O sistema deve gerir classificacoes economicas.
RF-009	Receitas	O sistema deve registar receitas e gerar RUPE automaticamente.
RF-010	Receitas	O sistema deve impedir receita retroactiva sem autorizacao.
RF-011	Auditoria	O auditor deve autorizar pedidos de receita retroactiva e gerar PDF.
RF-012	Despesas	O sistema deve registar despesas com estado financeiro.
RF-013	Despesas	O sistema deve permitir liquidar e pagar despesas conforme regras de estado.
RF-014	Dashboard	O sistema deve apresentar indicadores de tecto, receitas, despesas e risco.
RF-015	Relatorios	O sistema deve exportar relatorios em PDF e Excel.
RF-016	Auditoria	O sistema deve registar logs de operacoes relevantes.

2.2.2 Requisitos Nao Funcionais

N.o	Categoria	Descricao
RNF-001	Seguranca	Palavras-passe devem ser armazenadas com hash BCrypt.
RNF-002	Seguranca	Tokens de acesso devem ser enviados em cookies HttpOnly.
RNF-003	Seguranca	Refresh tokens persistidos devem ser guardados por hash.
RNF-004	Seguranca	A API deve aplicar RBAC no backend.
RNF-005	Integridade	O banco deve usar chaves estrangeiras, unicidade e checks.
RNF-006	Auditoria	Eventos criticos devem gerar logs persistentes.
RNF-007	Manutenibilidade	A aplicacao deve separar Controller, Service, Repository e DTO.
RNF-008	Usabilidade	A interface deve ser responsiva, clara e adequada a uso institucional.
RNF-009	Disponibilidade	O sistema deve possuir health check para monitorizacao.
RNF-010	Qualidade	O projecto deve possuir testes automatizados e build validado.

2.2.3 Requisitos de Interface

N.o	Descricao
RI-001	A interface deve ter identidade institucional, com cores tranquilas e boa legibilidade.
RI-002	O dashboard deve apresentar informacao financeira de forma escaneavel e profissional.
RI-003	Os formularios devem mostrar campos, erros e estados de carregamento com clareza.
RI-004	As tabelas devem suportar paginacao, pesquisa e leitura rapida.
RI-005	A navegacao deve adaptar-se aos perfis ADMIN, GESTOR e AUDITOR.
RI-006	A aplicacao deve funcionar adequadamente em desktop, tablet e mobile.

3 Metodologia

3.1 Tipo de Pesquisa

O projecto enquadra-se como pesquisa aplicada, pois procura resolver um problema pratico: a necessidade de uma plataforma digital para gestao e controlo das finanças publicas. A pesquisa também é descritiva, porque apresenta a solucao, os seus modulos, tecnologias, regras de negocio e resultados obtidos.

A abordagem adoptada é mista. A parte qualitativa surge na análise do dominio, identificacao dos fluxos financeiros e definicao dos perfis de utilizador. A parte quantitativa manifesta-se na validacao de totais financeiros, execucao de testes automatizados, verificacao de integridade da base de dados e resultados de build.

3.2 Universo e Amostra

O universo do sistema inclui entidades publicas, Unidades Orcamentais, administradores de sistema, gestores financeiros e auditores. Para fins academicos e de desenvolvimento, a amostra considerada corresponde aos principais perfis funcionais:

- administrador responsavel por UO, utilizadores, orcamentos e classificacoes;
- gestor responsavel por operacoes financeiras da sua UO;
- auditor responsavel por consulta, relatorios, logs e autorizacoes especiais.

3.3 Tecnicas de Recolha de Dados

Foram usadas as seguintes tecnicas:

- análise documental sobre processos de gestao financeira, orcamento, receitas, despesas e auditoria;
- levantamento de requisitos a partir do sistema SGFE existente e suas regras de negocio;
- observacao de fluxos funcionais de administracao, gestao e auditoria;
- estudo tecnico de frameworks, bibliotecas e boas praticas de seguranca;
- testes praticos de API, frontend, backend e base de dados.

3.4 Instrumentos de Recolha de Dados

Os principais instrumentos usados foram:

- documentos tecnicos do proprio projecto;
- migrations SQL e estrutura relacional da base de dados;
- codigo-fonte do backend e frontend;
- testes unitarios;
- validacoes manuais com chamadas HTTP;
- build de producao do frontend;
- execucao local com backend e frontend integrados.

3.5 Metodologia de Desenvolvimento

O desenvolvimento seguiu uma abordagem incremental, inspirada em metodologias ageis. O projecto foi dividido em fases:

1. levantamento e consolidacao das regras do dominio;
2. modelagem da base de dados relacional;
3. implementacao do backend em Spring Boot;
4. implementacao da autenticacao, autorizacao e auditoria;
5. implementacao do frontend em Next.js;
6. integracao com API REST;
7. melhorias de interface e experiencia de utilizador;
8. reforco de seguranca;
9. testes automatizados e validacao final.

Esta abordagem permitiu validar cada modulo separadamente e reduzir o risco de falhas em areas sensiveis.

3.6 Delimitacao do Tema

O projecto delimita-se ao desenvolvimento de uma plataforma web para gestao financeira publica em ambiente institucional. O SGFE cobre UO, utilizadores, orcamentos, classificacoes, receitas RUPE, despesas, relatorios, dashboard e auditoria. Nao abrange, nesta versao, integracoes oficiais com sistemas externos, assinatura digital, workflow completo de aprovacao multi-nivel, pagamentos bancarios reais ou ambiente cloud de producao.

4 Analise e Apresentacao dos Resultados

4.1 Caracterizacao do Projecto

4.1.1 Tipo de Projecto

O SGFE e um sistema de informacao web, de natureza institucional e administrativa, voltado para gestao financeira publica. A solucao possui frontend, backend, base de dados relacional, camada de seguranca, relatorios e auditoria.

4.1.2 Area de Aplicacao

A area de aplicacao e a gestao das finanças publicas, com foco em execucao orcamental, arrecadacao de receitas, gestao de despesas, relatorios e fiscalizacao.

4.1.3 Publico-Alvo

O publico-alvo inclui:

- administradores do sistema;
- responsaveis por Unidades Orcamentais;
- gestores financeiros;
- auditores;
- tecnicos de informatica responsaveis pela manutencao;
- entidades institucionais que necessitam de informacao financeira consolidada.

4.1.4 Problema Identificado

O problema identificado e a dificuldade de gerir processos financeiros de forma integrada, segura e auditavel. Sem uma plataforma adequada, os dados podem ficar dispersos, os perfis de acesso podem ser pouco claros e a auditoria pode depender de registos incompletos.

4.1.5 Solucao Proposta

A solucao proposta e uma plataforma web composta por:

- frontend moderno e responsivo;
- API REST segura;
- base de dados relacional;
- autenticacao e autorizacao por perfil;
- modulos de gestao financeira;
- dashboard operacional;
- relatorios exportaveis;
- auditoria persistente.

4.2 Missao, Visao e Valores

4.2.1 Missao

Oferecer uma plataforma digital segura e eficiente para apoiar a gestao, execucao e fiscalizacao das finanças publicas.

4.2.2 Visao

Tornar-se uma base tecnologica de referencia para controlo financeiro institucional, promovendo transparencia, rastreabilidade e eficiencia administrativa.

4.2.3 Valores

- transparencia;
- seguranca;
- integridade dos dados;
- responsabilidade institucional;
- usabilidade;
- confiabilidade;
- evolucao continua.

4.3 Organograma Funcional do Projecto

Coordenacao / Orientacao

```
|
+-- Analise de Requisitos
+-- Backend e Seguranca
+-- Frontend e Experiencia de Utilizador
+-- Base de Dados e Relatorios
+-- Testes, Validacao e Documentacao
```

4.4 Tecnologias e Ferramentas

Camada	Tecnologias
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS
Estado e dados	TanStack Query, TanStack Table
Formularios	React Hook Form, Zod
Graficos	Recharts
Interface	componentes reutilizaveis, icones lucide-react
Backend	Java 21, Spring Boot 3.3.7
Seguranca	Spring Security, JWT, BCrypt, cookies HttpOnly
Persistencia	Spring Data JPA, Hibernate
Base de dados	MySQL 8, InnoDB, utf8mb4
Migracoes	Flyway
Relatorios	OpenPDF, Apache POI
Testes	JUnit, Spring Security Test, TypeScript typecheck
Ambiente	Maven, npm, script de desenvolvimento local

4.5 Arquitectura do Sistema

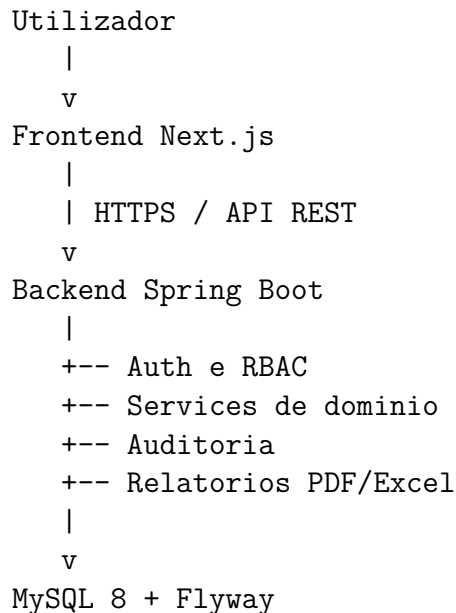
4.5.1 Visao Geral

O SGFE segue uma arquitectura cliente-servidor. O frontend comunica com o backend atraves de uma API REST. O backend concentra regras de negocio, seguranca, transacoes e persistencia.

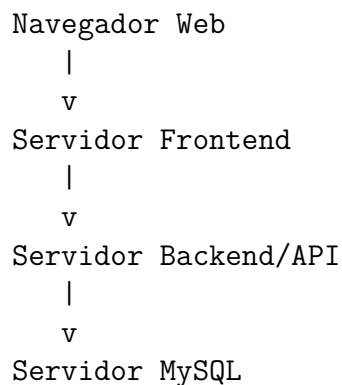
A base de dados MySQL guarda dados financeiros, utilizadores, tokens, logs e configuracoes.

4.5.2 Arquitectura Logica

Figura 1 - Arquitectura logica do SGFE



4.5.3 Arquitectura Fisica



Em ambiente local, o frontend é executado com `npm run dev` e o backend com `mvn spring-boot:run`. A configuração do backend aceita CORS apenas para origens autorizadas.

4.6 Modelo de Dados

O modelo relacional do SGFE organiza-se em três blocos:

- núcleo institucional e financeiro;
- autenticação e segurança;
- auditoria e rastreabilidade.

Figura 3 - Modelo relacional simplificado

instituicoes

```

|-- users
|-- orcamentos
|-- transacoes_receitas
|-- transacoes_despesas
|-- audit_logs

users
|-- refresh_tokens
|-- transacoes_despesas
|-- audit_logs

classificacoes_economicas
|-- transacoes_receitas
|-- transacoes_despesas

autorizacoes_receitas_retroativas
|-- instituicoes
|-- users solicitante
|-- users auditor
|-- transacoes_receitas

```

Tabela	Finalidade
instituicoes	cadastro de Unidades Orcamentais
users	utilizadores, perfis e estado
refresh_tokens	renovacao e revogacao de sessoes
password_reset_tokens	recuperacao de palavra-passe
classificacoes_economicas	catalogo de rubricas
orcamentos	tecto por UO e ano fiscal
transacoes_receitas	receitas RUPE
transacoes_despesas	despesas e estados financeiros
audit_logs	auditoria persistente
autorizacoes_receitas_retroativas	controlo de receitas com data anterior

5 Desenvolvimento dos Modulos do Sistema

5.1 Modulo de Gestao de Utilizadores e Identidade

5.1.1 Objectivo do Modulo

Este modulo controla autenticacao, sessao, recuperacao de palavra-passe, perfis, estado dos utilizadores e associacao a Unidades Orcamentais. O seu objectivo e garantir que apenas utilizadores autorizados acessem ao sistema e que cada acao seja executada de acordo com o perfil adequado.

5.1.2 Identificacao dos Actores

Actor	Responsabilidade
ADMIN	gere utilizadores, UO, orcamentos e configuracoes principais
GESTOR	opera receitas e despesas da sua Unidade Orcamental
AUDITOR	consulta informacao, analisa logs e autoriza receitas retroactivas

5.1.3 Fluxo de Autenticacao

Figura 2 - Fluxo de autenticacao e renovacao de sessao

Login

- > validacao de email e palavra-passe
- > criacao de access token JWT
- > criacao de refresh token seguro
- > armazenamento de hash do refresh token
- > envio em cookies HttpOnly

Refresh

- > validacao do refresh token
- > revogacao do refresh antigo
- > emissao de novo par de tokens

5.1.4 Implementacao

O backend utiliza Spring Security, AuthenticationProvider, BCryptPasswordEncoder, JwtAuthenticationFilter e servicos proprios de autenticacao. O refresh token e persistido como hash SHA-256, evitando armazenamento directo do segredo.

A interface de login usa chamadas seguras para a API, envia X-Requested-With e protege o redireccionamento pos-login contra caminhos externos maliciosos.

5.1.5 Tabela de Permissoes

Recurso	ADMIN	GESTOR	AUDITOR
Dashboard	Sim	Sim	Sim
UO	Sim	Nao	Consulta
Utilizadores	Sim	Nao	Nao
Orcamentos	Sim	Consulta do proprio tecto	Consulta
Classificacoes	Sim	Consulta	Consulta
Receitas	Sim	Propria UO	Consulta
Despesas	Sim	Propria UO	Consulta
Relatorios	Sim	Propria UO	Sim
Auditoria	Sim	Nao	Sim

5.2 Modulo de Unidades Orcamentais

O modulo de UO permite cadastrar, editar, listar e controlar entidades institucionais. Cada UO possui codigo unico, nome, tipo, responsavel e estado.

A criacao de uma UO pode ser acompanhada da criacao do utilizador responsavel, reduzindo o risco de entidades sem operador. O backend garante unicidade do codigo e protege remocoes quando existem vinculos financeiros.

5.3 Modulo de Orcamentos e Tecto Orcamental

Este modulo permite atribuir tecto orcamental por Unidade Orcamental e ano fiscal. A regra principal e que uma UO so pode possuir um orcamento por ano fiscal.

O controlo de saldo considera como comprometedoras as despesas nos estados:

- PENDENTE_CABIMENTADA;
- LIQUIDADA_APROVADA;
- PAGA.

Assim, o saldo disponivel resulta da diferenca entre o valor total do orcamento e o valor comprometido por despesas activas.

5.4 Modulo de Classificacoes Economicas

As classificacoes economicas funcionam como catalogo de rubricas financeiras. Elas sao usadas para organizar receitas e despesas, permitindo relatorios por natureza economica e melhor analise contabilistica.

Cada classificacao possui codigo unico, descricao e tipo. A existencia deste modulo evita texto livre desorganizado e melhora a qualidade dos dados financeiros.

5.5 Modulo de Receitas RUPE

5.5.1 Objectivo do Modulo

O modulo de receitas permite registar valores arrecadados, associando-os a uma UO, classificacao economica, fonte de receita e codigo RUPE.

5.5.2 Fluxo da Receita

Figura 4 - Fluxo da receita RUPE

Utilizador autorizado

- > preenche fonte, valor e classificacao
- > backend valida UO, data e valor
- > backend gera RUPE de 20 digitos
- > banco garante unicidade do RUPE
- > receita e gravada
- > operacao e auditada

5.5.3 Receita Retroactiva

Para preservar controlo temporal, receitas com data anterior exigem autorizacao. O fluxo e:

1. ADMIN ou GESTOR solicita autorizacao retroactiva;
2. pedido fica com estado PENDENTE;
3. AUDITOR analisa e autoriza;
4. sistema gera PDF da autorizacao;
5. autorizacao fica AUTORIZADA;
6. receita e criada usando a autorizacao;
7. autorizacao passa a UTILIZADA.

Este mecanismo impede que datas passadas sejam usadas livremente, criando trilha de auditoria.

5.6 Modulo de Despesas

5.6.1 Objectivo do Modulo

O modulo de despesas acompanha a execucao financeira desde a cabimentacao ate ao pagamento. O sistema valida valor, UO, classificacao e saldo disponivel.

5.6.2 Fluxo da Despesa

Figura 5 - Fluxo da despesa publica

Criacao da despesa

- > estado PENDENTE_CABIMENTADA
- > compromete tecto

Liquidacao

- > estado LIQUIDADA_APROVADA
- > confirma obrigacao

Pagamento

- > estado PAGA
- > encerra fluxo financeiro

O backend impede transicoes invalidas, como pagar uma despesa antes da liquidacao ou liquidar uma despesa ja paga.

5.7 Modulo de Dashboard

O dashboard apresenta uma visao operacional do sistema. Ele inclui:

- tecto orcamental;
- valor comprometido;
- valor pago;
- saldo disponivel;
- total de receitas;
- percentual de execucao;
- risco orcamental;
- top Unidades Orcamentais;
- indicadores visuais de controlo.

A interface foi redesenhada para parecer menos artificial e mais profissional, com menor agressividade visual, superficies tranquilas, contraste adequado e leitura rapida.

5.8 Modulo de Relatorios

O SGFE gera relatorios para apoio a decisao e auditoria. Os principais relatorios incluem:

- resumo financeiro em PDF;
- despesa por natureza em PDF;
- auditoria operacional em PDF;
- receitas RUPE em Excel.

Os relatorios sao gerados no backend para garantir seguranca, consistencia de dados e possibilidade de auditoria.

5.9 Modulo de Auditoria

O modulo de auditoria permite consultar logs de operacoes relevantes. Cada log pode conter:

- utilizador;
- Unidade Orcamental;
- accao;
- entidade afectada;
- resultado;
- severidade;
- endereco IP;
- user agent;
- correlation id;
- contexto em JSON;
- data e hora.

Esta abordagem permite responder a perguntas fundamentais: quem fez, quando fez, em que entidade, com que resultado e em que contexto.

5.10 Modulo de Seguranca

O SGFE aplica varias medidas de seguranca:

- BCrypt para palavras-passe;
- JWT em cookies HttpOnly;
- refresh token com hash e rotacao;
- RBAC no backend;
- CORS com origens permitidas;
- cabecalhos CSP, HSTS, X-Frame-Options, Referrer-Policy e Permissions-Policy;
- protecao contra pedidos suspeitos sem intencao explicita de API;
- validacao de secret JWT e duracoes de sessao;
- respostas JSON controladas para 401 e 403;
- escopo institucional por perfil.

5.11 Estudo de Viabilidade

O SGFE e viavel do ponto de vista tecnico porque usa tecnologias maduras, documentadas e amplamente adoptadas. A arquitectura em camadas facilita manutencao e evolucao. O uso de MySQL com FKs, indices e constraints reforca a integridade do dominio financeiro.

Do ponto de vista institucional, a plataforma reduz dependencia de processos manuais, melhora transparencia e cria base para relatorios consistentes. A existencia de perfis e logs permite separar responsabilidades e aumentar a confianca operacional.

Do ponto de vista economico, a solucao aproveita tecnologias open-source e pode ser implantada inicialmente em infraestrutura local ou em ambiente cloud, conforme disponibilidade da instituicao.

5.12 Resultados Obtidos

Foram obtidos os seguintes resultados:

- frontend Next.js funcional;
- backend Spring Boot funcional;
- base de dados MySQL versionada por Flyway;
- autenticacao com JWT e refresh token;
- gestao de UO, utilizadores, orcamentos e classificacoes;
- receitas RUPE e autorizacao retroactiva;
- despesas com estados financeiros;
- dashboard profissional;
- relatorios PDF/Excel;
- auditoria persistente;
- reforco de seguranca;
- testes automatizados no backend;
- typecheck e build do frontend validados.

6 Conclusao

O SGFE demonstrou que e possivel construir uma plataforma moderna, segura e funcional para apoiar a gestao das finanças publicas. O sistema centraliza Unidades Orcamentais, utilizadores, orcamentos, classificacoes, receitas, despesas, relatorios e auditoria, reduzindo a fragmentacao da informacao e melhorando a rastreabilidade das operacoes.

A separacao entre frontend, backend e base de dados permitiu criar uma solucao mais organizada e manutenivel. O frontend facilita a utilizacao diaria, enquanto o backend concentra as regras criticas de seguranca e negocio. A base de dados relacional garante integridade, unicidade e preservacao historica.

Os objectivos definidos foram atingidos. O sistema implementa perfis de acesso, autenticacao segura, regras financeiras, dashboard, relatorios e auditoria. Alem disso, foram aplicadas melhorias recentes de interface e seguranca, tornando a plataforma mais profissional e preparada para apresentacao.

Conclui-se que o SGFE nao e apenas um exercicio academico. Trata-se de uma proposta concreta de informatizacao financeira institucional, com potencial para evoluir e integrar processos mais avancados de controlo, aprovacao, fiscalizacao e tomada de decisao.

7 Perspectivas Futuras

Area	Evolucao proposta
Segregacao de funcoes	Separar perfis para cabimentacao, liquidacao e pagamento.
Assinatura digital	Integrar aprovacao documental com certificado digital.
Notificacoes	Enviar alertas para despesas acima de limite, autorizacoes pendentes e riscos.
BI	Criar dashboards avancados para analise historica e comparativa.
OpenAPI	Documentar a API automaticamente para integracoes.
Integracoes oficiais	Integrar com sistemas externos de tesouraria, contabilidade e pagamentos.
Auditoria avancada	Adicionar trilha de alteracoes antes/depois por entidade.
Deploy	Criar pipeline CI/CD e ambiente de staging.
Backup	Automatizar copias de seguranca e testes de restauracao.
Acessibilidade	Aprofundar contraste, navegacao por teclado e compatibilidade com leitores de ecra.

Tambem se recomenda implementar testes end-to-end dos fluxos principais: login, criacao de UO, criacao de orcamento, receita RUPE, autorizacao retroactiva, despesa, liquidacao, pagamento, relatorio e consulta de auditoria.

8 Referencias Bibliograficas

Apache Software Foundation. (2026). Apache POI Documentation. Disponivel em: <https://poi.apache.org/>

Flyway. (2026). Flyway Documentation. Disponivel em: <https://documentation.red-gate.com/fd>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

MySQL. (2026). MySQL 8.0 Reference Manual. Disponivel em: <https://dev.mysql.com/doc/>

Next.js. (2026). Next.js Documentation. Disponivel em: <https://nextjs.org/docs>

OWASP Foundation. (2026). OWASP Application Security Verification Standard. Disponivel em: <https://owasp.org/>

React. (2026). React Documentation. Disponivel em: <https://react.dev/>

Spring. (2026). Spring Boot Reference Documentation. Disponivel em: <https://docs.spring.io/spring-boot/>

Spring. (2026). Spring Security Reference Documentation. Disponivel em: <https://docs.spring.io/spring-security/>

TanStack. (2026). TanStack Query and Table Documentation. Disponivel em: <https://tanstack.com/>

Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development*. Addison-Wesley.

9 Apendices

9.1 Apendice I - Endpoints Principais da API REST

Metodo	Endpoint	Perfil	Funcao
POST	/api/auth/login	Publico	autenticar utilizador
POST	/api/auth/refresh	Publico com cookie	renovar sessao
POST	/api/auth/logout	Autenticado	terminar sessao
POST	/api/auth/forgot-password	Publico	solicitar recuperacao
POST	/api/auth/reset-password	Publico	redefinir palavra-passe
GET	/api/auth/me	Autenticado	obter utilizador actual
GET	/api/users	ADMIN	listar utilizadores
POST	/api/users	ADMIN	criar utilizador
PUT	/api/users/{id}	ADMIN	editar utilizador
PATCH	/api/users/{id}/role	ADMIN	alterar perfil/estado
GET	/api/users/me	Autenticado	perfil actual
PUT	/api/users/me	Autenticado	editar perfil
PATCH	/api/users/me/password	Autenticado	alterar palavra-passe
GET	/api/instituicoes	ADMIN/AUDITOR	listar UO
POST	/api/instituicoes	ADMIN	criar UO
PUT	/api/instituicoes/{id}	ADMIN	editar UO
DELETE	/api/instituicoes/{id}	ADMIN	remover UO
GET	/api/orcamentos	ADMIN/AUDITOR	listar orcamentos
GET	/api/orcamentos/meu	ADMIN/GESTOR	consultar tecto
POST	/api/orcamentos	ADMIN	criar orcamento
PUT	/api/orcamentos/{id}	ADMIN	editar orcamento
DELETE	/api/orcamentos/{id}	ADMIN	remover orcamento
GET	/api/classificacoes	Todos	listar classificacoes
POST	/api/classificacoes	ADMIN	criar classificacao
PUT	/api/classificacoes/{id}	ADMIN	editar classificacao
DELETE	/api/classificacoes/{id}	ADMIN	remover classificacao
GET	/api/receitas	Todos	listar receitas
POST	/api/receitas	ADMIN/GESTOR	criar receita
DELETE	/api/receitas/{id}	ADMIN/GESTOR	remover receita
GET	/api/receitas/autorizacoes-retroativas	Todos	listar autorizacoes
POST	/api/receitas/autorizacoes-retroativas	ADMIN/GESTOR	solicitar autorizacao
POST	/api/receitas/autorizacoes-retroativas/{id}/autorizar	AUDITOR	autorizar
GET	/api/receitas/autorizacoes-retroativas/{id}/pdf	Todos	gerar PDF
GET	/api/despesas	Todos	listar despesas
POST	/api/despesas	ADMIN/GESTOR	criar despesa
POST	/api/despesas/{id}/liquidar	ADMIN/GESTOR	liquidar despesa
POST	/api/despesas/{id}/pagar	ADMIN/GESTOR	pagar despesa
DELETE	/api/despesas/{id}	ADMIN/GESTOR	remover despesa
GET	/api/dashboard	Autenticado	indicadores
GET	/api/auditoria/logs	ADMIN/AUDITOR	consultar logs
GET	/api/relatorios/expedientes/resumo-financeiro	Todos	PDF financeiro

Metodo	Endpoint	Perfil	Funcao
GET	/api/relatorios/expo	Todos	PDF despesas
GET	/api/relatorios/expo	ADMIN/AUDITOR	PDF funcional
GET	/api/relatorios/expo	Todos	Excel receitas

9.2 Apendice II - Variaveis de Ambiente

9.2.1 Backend

```
SGFE_DB_URL=jdbc:mysql://localhost:3306/sgfe?useUnicode=true&characterEncoding=utf8&
SGFE_DB_USERNAME=sgfe_user
SGFE_DB_PASSWORD=sgfe_pass
SGFE_JWT_SECRET=change-this-development-secret-change-this-development-secret
SGFE_ACCESS_TOKEN_MINUTES=15
SGFE_REFRESH_TOKEN_DAYS=7
SGFE_COOKIE_SECURE=false
SGFE_COOKIE_SAME_SITE=Lax
SGFE_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
SGFE_PORT=8080
SGFE_BOOTSTRAP_ADMIN_EMAIL=admin@sgfe.gov.ao
SGFE_BOOTSTRAP_ADMIN_PASSWORD=
SGFE_BOOTSTRAP_TEST_DATA=false
```

9.2.2 Frontend

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:8080/api
```

9.3 Apendice III - Instalacao e Arranque

9.3.1 Backend

```
cd backend
mvn spring-boot:run
```

9.3.2 Frontend

```
cd frontend
npm install
npm run dev
```

9.3.3 Arranque integrado

```
./scripts/dev.sh
```

Se a porta 8080 estiver ocupada:

```
SGFE_PORT=18080 ./scripts/dev.sh
```

9.4 Apendice IV - Validacao e Testes

Comandos usados para validacao:

```
cd backend  
mvn test
```

Resultado obtido: 7 testes executados, 0 falhas, 0 erros.

```
cd frontend  
npm run typecheck  
npm run build
```

Resultado obtido: typecheck concluido e build de producao gerado com sucesso.

Tambem foram validados:

- pagina /login com resposta HTTP 200;
- pagina inicial com resposta HTTP 200;
- health check do backend com {"status": "UP"};
- CORS do login com origem autorizada;
- chamada real de login invalido com resposta 401 controlada;
- cabecalhos de seguranca no frontend e backend.

9.5 Apendice V - Estrutura do Projecto

```
SGFE/  
  backend/  
    src/main/java/ao/gov/minfin/sgfe/  
      auth/  
      users/  
      instituicoes/  
      orcamentos/  
      classificacoes/  
      receitas/  
      despesas/  
      dashboard/  
      relatorios/  
      auditoria/  
      common/  
    src/main/resources/db/migration/  
  frontend/  
    app/  
    components/  
    lib/  
    public/  
  database/  
  docs/  
  scripts/
```

9.6 Apendice VI - Checklist de Prontidao

Item	Estado
Autenticacao funcional	Concluido
RBAC no backend	Concluido
Cookies HttpOnly	Concluido
Refresh token com hash	Concluido
Gestao de UO	Concluido
Gestao de utilizadores	Concluido
Orcamentos	Concluido
Classificacoes	Concluido
Receitas RUPE	Concluido
Autorizacao retroactiva	Concluido
Despesas	Concluido
Dashboard	Concluido
Relatorios	Concluido
Auditoria	Concluido
Testes backend	Concluido
Typecheck frontend	Concluido
Build frontend	Concluido
Headers de seguranca	Concluido
CORS local	Concluido
Documentacao final	Concluido